VDK/TLM Creation

Rapid Training

X-2025.06

Module 1: Lab C

Implement the Behavior of an Interrupt Timer

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview *2*

Task-1: Open the Timer Project **2**

Task-2: Implement the Set-And-Forget Mode Timer Behavior **3**

Task-3: Adding a Unit test for the Interrupt Flag Clearing Behavior (Optional task) **8**

Task-4: What's Next **10**

Lab Overview

Your task for this lab is to the implement the `set-and-forget` behavior of the timer model as described in the specification of the interval timer and as listed in the feature tracking spreadsheet `TimerFunctionalityTracking.xlsx`.

During this lab, you will see how to create TLM models in an efficient way, using TLM Creator's productivity features such as callbacks, continuous assignments, a library of re-use objects, etc.

Task-1: Open the Timer Project

- ***Motivation**: We have already created a shell of a programmable interval timer model, including a basic unit testbench, and basic register access and reset behavior in Lab A and Lab B.*

1. If you have completed the previous lab (Module 1 Lab B), including the optional task of creating a unit test for the `set-and-forget` behavior, you can just open Virtualizer Studio and continue with the same `timer` TLMC project and skip this task. Continue with Task-2.

   If have not completed the previous lab or want to start from a prepared starting point for this lab, use the following steps to create the starting point.

   i. Open a console and invoke Virtualizer studio from the Synopsys Virtual Prototyping installation.

      ```
      % source <installDir>/SLS/windows/setup.bat vi[rtualizer] (Windows)

      % source <installDir>/SLS/linux/setup.csh|sh vi[rtualizer] (Linux)
      ```

      This sourcing option picks up the Virtualizer *Elite* or *Base* license in that order, whichever is available. For more details on other options, refer to the VP documentation.

   ii. Go to the `<Lab-data>/data/module_1` directory and start Virtualizer Studio.

      ```
      vs -d start_Module1LabC &
      ```

2. Close the welcome screen and import the starting point TLM Creator Project by clicking ***File*** → ***Import…*** menu option.

   i. In the ***Import*** dialog, select ***General*** → ***Existing Projects into Workspace*** to select what you want to import.

   ii. Click ***Next*** to get to the ***Import Projects*** dialog. Browse to the `timer` directory in the `start_Module1LabC` directory and click ***Finish*** (Make sure you are in the right directory – it's easy to end up in the wrong directory!).

   iii. Select the `timer.tlmc` file in the Project Explorer and click ***Generate Sources Code*** from the ***Build*** toolbar button options. Also, generate the documentation using the ***Generate Documentation*** option.

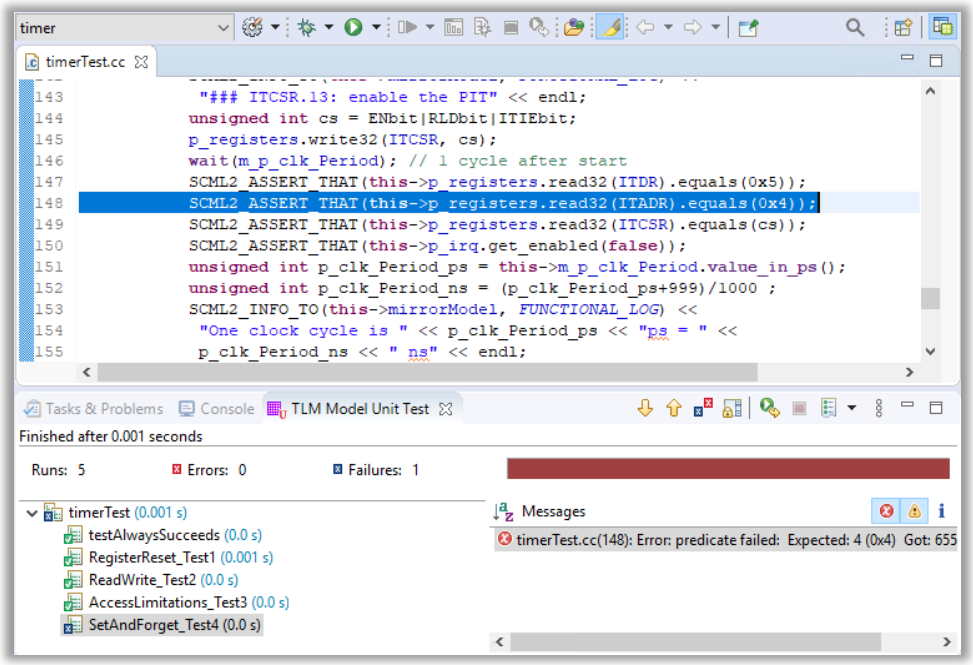
This will provide you with the starting point `timer` model for this lab.

Task-2: Implement the Set-And-Forget Mode Timer Behavior

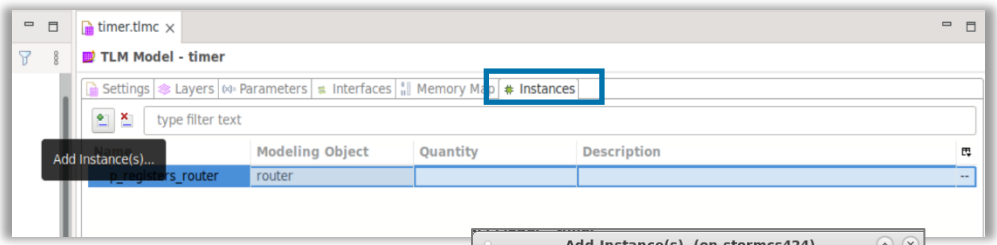
- ***Motivation:*** *In this task, we will add the behavior for the set-and-forget mode to the timer model. The unit test for this behavior should already be in place. So, let's concentrate on the implementation in this task.*

3. Run the unit tests for the timer model by clicking the ***Test Run*** ( )button from the toolbar. As expected, the unit test for the `set-and-forget` behavior fails with one of the first assertions as follows.

   The test expects the timer counter to be active and count down the ticks, so reading the *ITADR* register one cycle after start should give you the modulus value (*ITDR* = `0x5`) reduced by 1, i.e. `0x4`. It's no wonder that this does not work yet, since the corresponding behavior is not implemented yet.



4. To implement the timer behavior, you can make use of the re-use object library in TLM Creation perspective. Switch to the ***Instances*** page of the ***timer.tlmc*** project and create a new re-use object using the following steps.



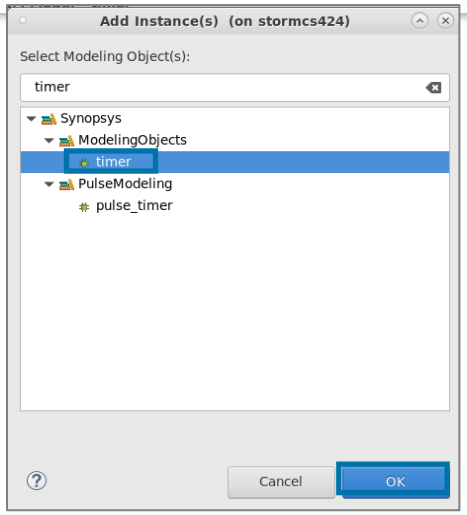
   i. Click the ***Add Object*** icon ( ) in the top-right corner.

      The Add Instance(s) dialog appears.

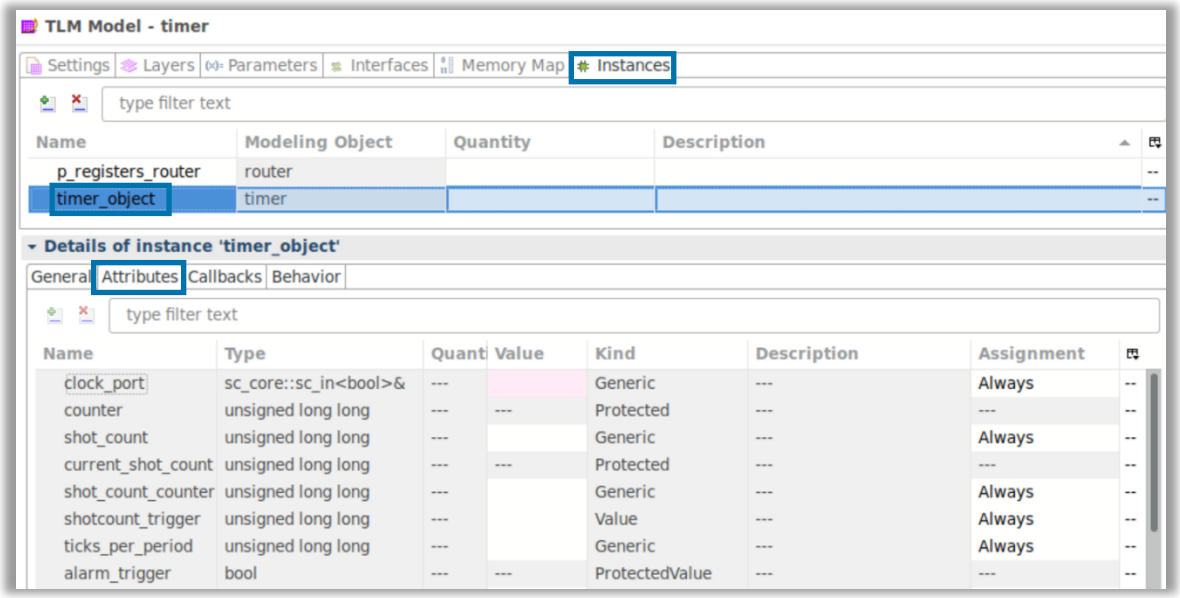
   ii. Type *timer* in the ***Select Modeling Object(s)*** field to look for the timer object in the Reuse object library.

   iii. Select `timer` instance under `ModelingObjects` library and click ***OK***.

   iv. Rename the object as *timer_object*. In the ***Context Sensitive Help*** window, you can get a detailed description of the timer reuse object, its attributes and behavior.



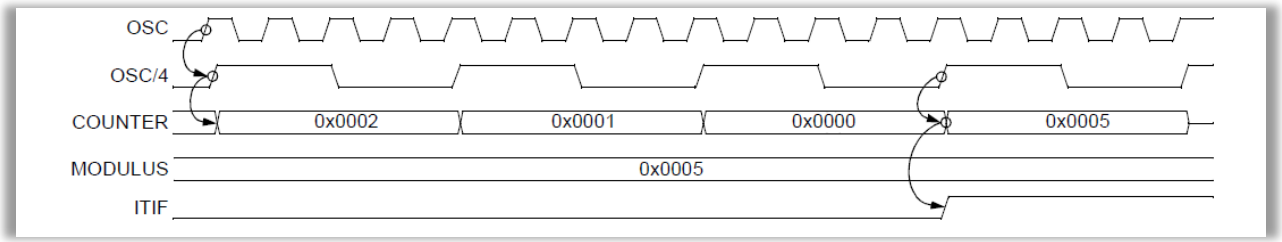
5. Next, configure the attributes of the timer reuse object to match the properties of our programmable interrupt timer model. Click to open the ***Attributes*** page and specify the following in the ***Value*** field.



v. For *clock_port,* specify *p_clk* (note that the red color disappears after you set the clock port).

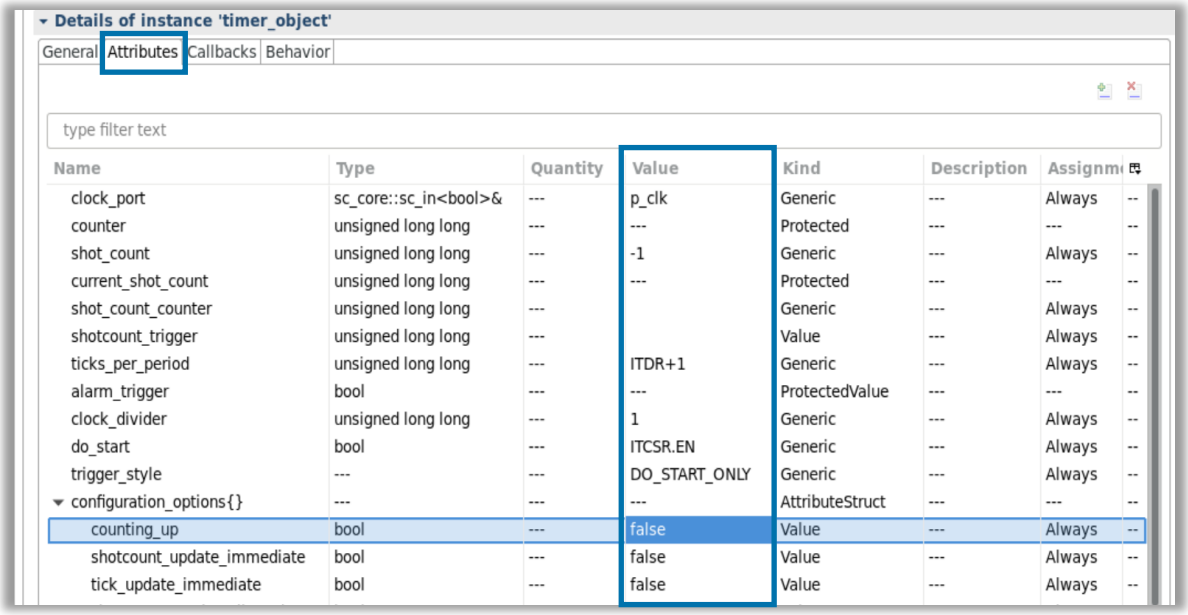
vi. For *shot_count,* enter *the value as -1* as per timer object documentation this value can be used to have the timer loop forever.

i. For the *ticks_per_period* attribute, enter the value as *ITDR+1*. The *+1* term is required because as per the specification of the PIT, the number of ticks per period is actually the value of the modulus register (*ITDR*) + 1, since the *ITIF* flag is only set ***after*** counting down from the modulus value to `0x0`. After that the counter is reloaded with the value of the *ITDR* register again:



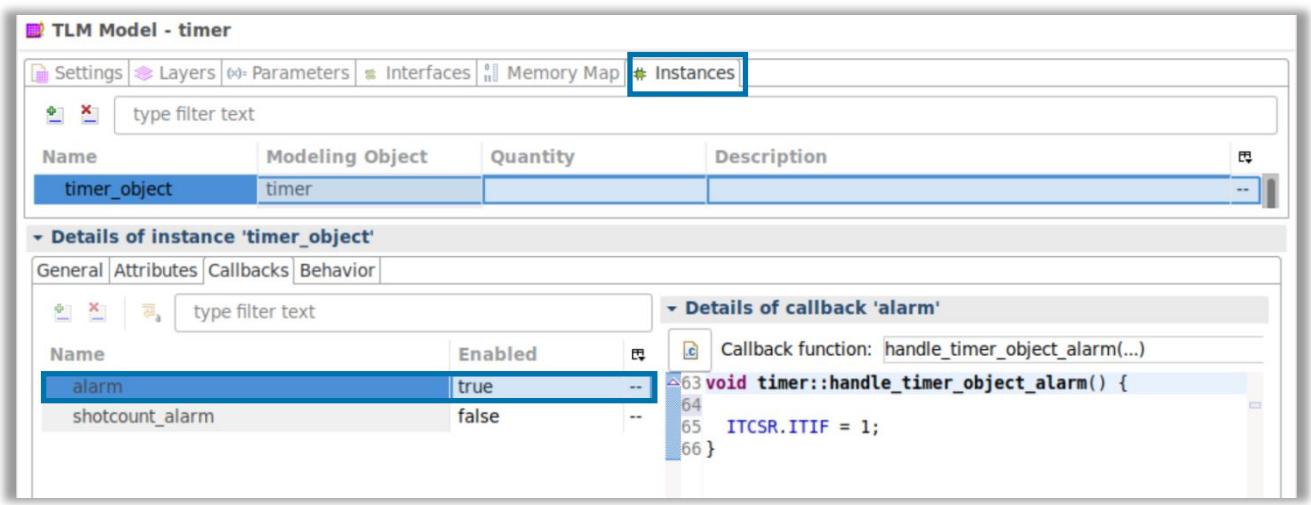
ii. Leave the *trigger_style* attribute with the `DO_START_ONLY` value (which means we will use the *do_start* attribute to start the timer) and lastly, set the value of *do_start* attribute as *ITCSR.EN*, i.e. the enable flag of the control/status register. The ***Assignment*** type `Always` indicates that this is a continuous assignment, i.e. whenever *ITCSR.EN* changes, this assignment is evaluated again.
Modeling the same manually, for e.g. by to writing an extra process that is sensitive to changes of the *ITCSR.EN* flag would be much more tedious and error prone.

iii. Unfold the ***configuration_options*** hierarchy and change the ***counting_up*** configuration from `true` to `false` (since the specification of the interval timer also describes a down-counting timer).



6. In the ***Context Help,*** you can see that the timer object has a callback `alarm()` that is fired when the timer expires – you can use this callback to set the *ITCSR.ITIF* flag: Switch to the ***Callback*** tab of the ***Details of instance 'timer object'*** section and set the ***Enabled*** field of the alarm callback to `true`.

7. Save the project to generate the default implementation of the `alarm` callback – then replace the `//FIXME` comment in the default implementation with the following assignment.



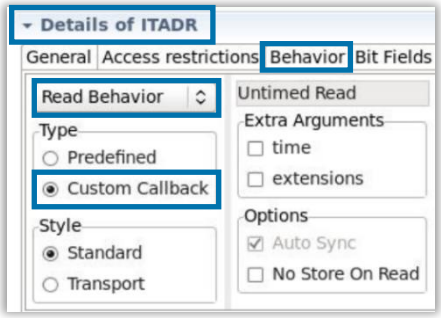

```
ITCSR.ITIF = 1;
```

8. Next you need to link the read behavior of the *ITADR* register to the counter value of the timer object.

   To implement the ***read behavior*** of the *ITADR* register, switch to the ***Memory Map*** page, select *ITADR* register in the table and then its ***Behavior*** tab in the ***Details of ITADR*** section. Here you should already have a default implementation of an untimed read callback, that you can replace by the following code.

```
bool timer::handle_read_ITADR(unsigned int& value, const unsigned int& byteEnables) {
  if ( byteEnables != 0xFFFFFFFF ) {
    SCML2_INFO(FUNCTIONAL_LOG) << "only 32bit access allowed - " <<
         "disabling access (byteEnables= " << byteEnables << ")" << endl;
  } else {
    int counter = timer_object.counter-1;
    SCML2_MODEL_INTERNAL(LEVEL4) << " counter = " << counter << endl;
    if ( timer_object.is_active() ) {
      value = counter;
    } else {
      value = ITDR;
    }
  }
   return true;
}
```

   Note: First time you select the ***Behavior*** tab you might not see the (correct) callback function and you first need to either select the ***Read Behavior*** or ***Custom Callback*** in the ***Behavior*** tab.



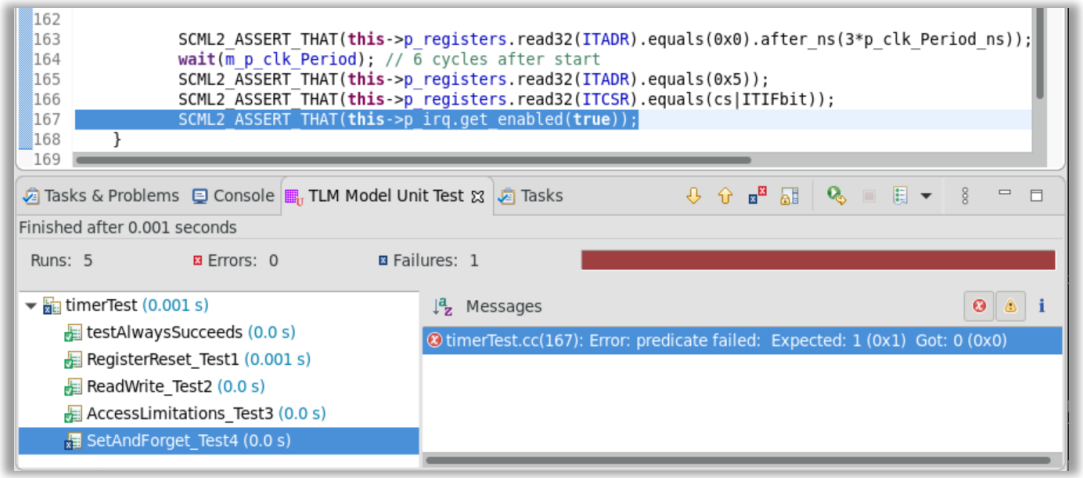
   To explain the behavior implementation done above, refer to the following points. This is not really a one-to-one mapping:

   i. The `timer_object.ticks_per_period` was set to `ITDR+1` so the timer counts down from `6` to `1`, while the PIT spec needs the counter to go from `5` to `0`. Hence the counter value you need here is `timer_object.counter-1`.

   ii. The `counter` attribute of the reuse object only takes the value of the *ITDR* register if it is active. If the *ITCSR.EN* register is false, the *ITADR* register should have the same content as the ITDR register.

   iii. Like the *ITDR* register the *ITADR* register also only allows 32-bit word accesses – every other read attempt should fail and return `0x0`.

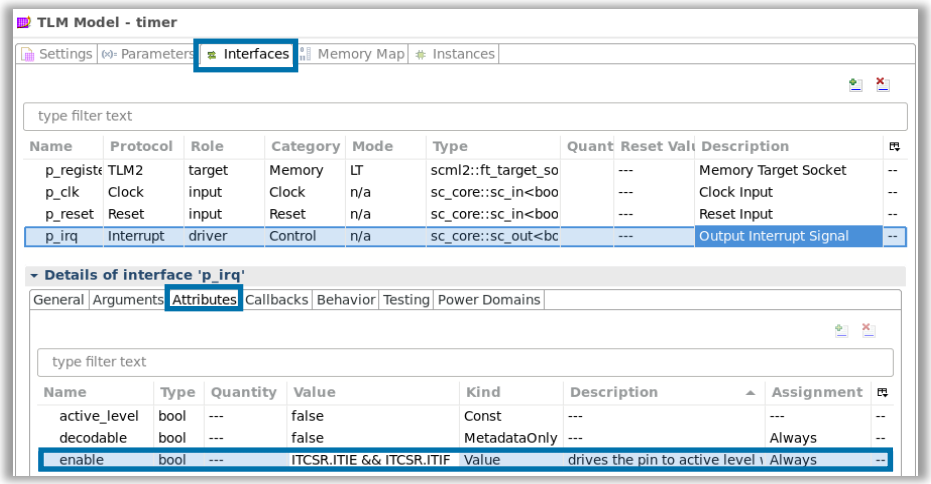
9. To check what you've implemented so far, save and build the model, and re-run the unit tests again by pressing the ***Test Run*** ( ) button



You can see that most of the assertions are passing now – the only thing that still fails is that the interrupt signal is not asserted after the timer has expired. That failure does not come as a surprise either, as we have not implemented the assertion of the interrupt signal yet.

10. To implement the assertion of the interrupt pin *p_irq*, select the ***Interfaces*** page in the ***timer.tlmc*** project, select the *p_irq* entry in the table, and switch to the ***Attributes*** tab in the ***Details*** section. Here, set the *enable* attribute ***Value*** (leaving the ***Assignment*** type as `Always`) to

    `ITCSR.ITIE && ITCSR.ITIF`



    That is, the interrupt is enabled through a continuous assignment – whenever the interrupt enable flag or the interrupt flag change their value, the expression is re-evaluated and reassigned. With this, the implementation of the features tested via the `SetAndForget_Test4` unit test is complete. Save the project.

11. Build your project again and run the unit tests. They should be all passing now.

12. And now that you have implemented the interrupt behavior, which exercises the p_irq interface, this should also cover the last untested/unreviewed coverpoint: Select the ***Functional coverage report*** and press the ***Refresh*** button to make sure you see the latest results:

Result	In Progress		
# Coverpoints to be reviewed	0	Reviewed Coverage:	100%
# Coverpoints to be tested	7	Tested Coverage:	96.6667%
Exempt # Coverpoints	3		
Total # Coverpoints	14		

Memory Exempted Coverpoints	Untested Unsafe
r_registers.ITCSR.OVW	
r_registers.ITCSR.ITIE	
r_registers.ITCSR.ITIF	
r_registers.ITCSR.RLD	
r_registers.ITCSR.EN	
r_registers.ITDR	
r_registers.ITADR	

Memory Exempted Coverpoints	Unmodeled Functional
r_registers.ITCSR.STOP	Not required for the Virtual Prototype project
r_registers.ITCSR.DOZE	Not required for the Virtual Prototype project
r_registers.ITCSR.DBG	Not required for the Virtual Prototype project

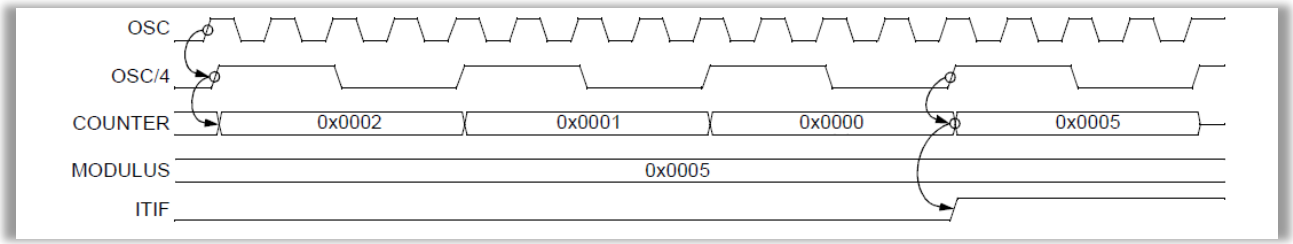
Note that the Result status has changed now from `Failed` to `In Progress`, as there are no unreviewed coverpoints any more i.e. Review Coverage is 100%. The Tested Coverage is not at 100% yet, so the test process is not completed yet.

Task-3: Adding a Unit test for the Interrupt Flag Clearing Behavior (Optional task)

- ***Motivation:*** *In the next lab, we will implement and debug parts of the interrupt flag clearing behavior of the timer as described in the timer functionality tracker (section 9.6.5 ITCSR). The starting point for the next lab includes a unit test for the behavior.*
  Instead of using the unit test in the starting point of Module 1 Lab D, you can write your own unit test in this task. We will provide you with few useful suggestions and hints.

13. Create and register a new unit test `ClearInterrupt_Test5`. The behavior to be tested is described in the spec and also listed in the Excel tracking sheet:
    i. `IT_Mode.2`: writing to `ITDR` sets the modulus of timer.
    ii. `ITCSR.7`: `ITIF` cleared when writing to the PIT data register `ITDR`.
    iii. `ITCSR.5`: direct overwrite of counter value only if `OVW` is 1.
14. To set the stage for the tests above, first drive the timer into a state where the interrupt flag *ITIF* is set:
    i. Write a modulus value (lets say:. `0x5`) to the `ITDR` register.
    ii. Next you need to write to the `ITCSR` register to set the `RLD`, `ITIE`, and `EN` bits to 1.
    iii. Assert that right after that the `ITIF` bit is not set and the interrupt *p_irq* is not enabled.

iv. Wait for 6 clock periods and assert that now the `ITIF` bit is set and the interrupt `p_irq` is



enabled, and the `ITADR` (counter) register shows the reloaded modulus value of `0x5` again.

15. Next, wait for one cycle and write a new value (let's say `0x8`) into the `ITDR` register and assert that

    i. The `ITCSR` registers only has only the `RLD`, `ITIE`, and `EN` bits set, not the `ITIF` bit any more (and also not the `OVW` bit).

    i. The interrupt `p_irq` is disabled.

16. Wait one more cycle (now it's 2 cycles after the timer started over) and assert that the `ITADR` register, representing the current counter value, carries the previous modulus value `0x5` (that was active when the timer started over) reduced by 2, i.e. `0x3`. If the `OVW` bit is unset the new modulus value is supposed to be only taken into account the next time the timer expires and starts over again.

ClearInterrupt_Test5 code for your reference

```
void ClearInterrupt_Test5() {
	SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
		"### IT_Mode.2: writing to ITDR sets the modulus of timer" << endl;
	p_registers.write32(ITDR, 0x5);
	SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
		"### SF.1: SF operation mode is selected when RLD is 1" << endl;
	SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
		"### SF.4: if the ITIE is set then  interrupt pin is fired" << endl;
	SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
		"### ITCSR.12: enable the PIT" << endl;
	unsigned int cs = ENbit|RLDbit|ITIEbit;
	p_registers.write32(ITCSR, cs);
	SCML2_ASSERT_THAT(this->p_registers.read32(ITDR).equals(0x5));
	SCML2_ASSERT_THAT(this->p_registers.read32(ITADR).equals(0x5));
	SCML2_ASSERT_THAT(this->p_registers.read32(ITCSR).equals(cs));
	SCML2_ASSERT_THAT(this->p_irq.get_enabled(false));
	wait(6*m_p_clk_Period); // 6 cycles after start)
	SCML2_ASSERT_THAT(this->p_registers.read32(ITADR).equals(0x5));
	SCML2_ASSERT_THAT(this->p_registers.read32(ITCSR).equals(cs|ITIFbit));
	SCML2_ASSERT_THAT(this->p_irq.get_enabled(true));
	wait(m_p_clk_Period);
	// 1 cycle after starting over
	SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
		"### IT_Mode.2: writing to ITDR sets the modulus of timer" << endl;
	SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
		"### ITCSR.7: ITIF cleared when writing to the PIT data register" << endl;
	p_registers.write32(ITDR, 0x8);
	SCML2_ASSERT_THAT(this->p_registers.read32(ITCSR).equals(cs)); // also asserts OVW==0
	SCML2_ASSERT_THAT(this->p_irq.get_enabled(false));
	SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
		"### ITCSR.5: direct overwrite of counter value only if OVW is 1" << endl;
	wait(m_p_clk_Period);
	// 2 cycles after starting over
	SCML2_ASSERT_THAT(this->p_registers.read32(ITADR).equals(0x3));
}
```

Task-4: What's Next

In the next lab we will highlight useful debugging techniques, including FastTrack, source code debugging, and SystemC debugging using VDK Debug features.